

UNIVERSIDAD DE CUNDINAMARCA

Facultad de Ingeniería · Sede Fusagasugá

Programa académico: Ingeniería de Sistemas y Computación

Asignatura: Ingeniería de Software I

FusaRoute

Sistema de información de transporte público para Fusagasugá y la región del Sumapaz

Entregable: Documento V2 (versión ampliada y corregida del Documento V1 del 16 de septiembre de 2026)

Integrantes del equipo

Santiago Bermúdez Gómez

Angélica María Aranguren Rozo

Docente: Ing. Luiferney Ortiz Parra

Semestre académico: 2026-2 (quinto semestre)

Fecha de entrega: 28 de septiembre de 2026

Tabla de contenido

1. Introducción del proyecto

2. Descripción general del sistema

2.1 Problema identificado · 2.2 Objetivo del sistema · 2.3 Usuarios del sistema · 2.4 Alcance del proyecto

3. Enfoque metodológico del proyecto

3.1 Análisis del contexto · 3.2 Enfoque metodológico seleccionado · 3.3 Uso de Scrum como marco de trabajo

4. Identificación y clasificación de requerimientos

4.1 Requerimientos funcionales · 4.2 Requerimientos no funcionales · 4.3 Priorización de requerimientos

5. Dependencias del proyecto

6. Configuración tecnológica del proyecto

7. Riesgos del proyecto

8. Diseño de base de datos

8.1 Modelo entidad-relación (MER) · 8.2 Modelo relacional · 8.3 Diccionario de datos

9. Conclusiones del proyecto

Anexo A. Registro de cambios V1 → V2

Anexo B. Arquitectura del sistema

Anexo C. Historias de usuario, casos de uso y correspondencia con los requerimientos

Anexo D. Contrato de la API

Anexo E. Métricas de calidad

Anexo F. Estado de avance, problemas y próximos pasos

Referencias

Las secciones 1 a 9 siguen la estructura pedida en el Entregable Documento V1. El material adicional (arquitectura, casos de uso, API, métricas y estado real de avance) va en los anexos para no alterar esa estructura.

1. Introducción del proyecto

Para nuestro proyecto queremos resolver la falta de información clara sobre las rutas de transporte público en Fusagasugá. Hoy hay más de veinte rutas urbanas de busetas y es difícil encontrar mapas actualizados o digitales que digan por dónde pasan; ni siquiera Google Maps las tiene registradas. Un habitante o un turista que quiere llegar a un destino puede hacerlo por varias rutas y, sin información, pierde tiempo y dinero eligiendo. La necesidad la respalda una encuesta propia del equipo (2025) y una población regional estimada en unos 285.000 habitantes (DANE).

Por eso desarrollamos FusaRoute, un sistema de información web: un backend que expone una API REST y un frontend de una sola página que la consume. Permite ver el catálogo de rutas, con sus barrios y tarifas, y, en las fases siguientes, sugerir la ruta de menor tiempo entre dos puntos. Se usa principalmente desde el celular o el computador, con conexión a internet, en el contexto urbano de Fusagasugá y en las rutas que la conectan con Chinauta, Pasca y Arbeláez (región del Sumapaz).

El objetivo general del proyecto es organizar la información de movilidad en un solo lugar, clara y actualizable por un administrador, para que los usuarios puedan planear mejor sus viajes y para que la aplicación no quede obsoleta. Este semestre entregamos el primer incremento funcional (cuenta de usuario y catálogo público de rutas) sobre una arquitectura hexagonal pensada para crecer; el estado real de avance está en el Anexo F.

2. Descripción general del sistema

2.1 Problema identificado

El problema principal es que no existe una fuente de datos oficial y abierta sobre las rutas de transporte público en la zona. La información vive en la práctica de quienes usan las busetas a diario, y la gente depende de preguntar o de mapas antiguos, lo que genera confusión, especialmente para quienes no conocen bien el municipio. Además, en Fusagasugá no hay paradas fijas (la buseta se para con la mano), así que ni siquiera hay un dato de paradas que consultar.

2.2 Objetivo del sistema

Con FusaRoute buscamos crear una herramienta donde se puedan ver las rutas, saber qué busetas pasan por dónde, cuánto cuesta el pasaje y cuál conviene tomar entre un origen y un destino, y donde el administrador pueda mantener esa información actualizada para que no quede obsoleta.

2.3 Usuarios del sistema

Identificamos tres tipos de usuarios:

Tipo de usuario	Descripción
Visitante	Cualquier persona, sin cuenta. Explora el catálogo público de rutas: listado, barrios y tarifa de cada una.
Usuario	Persona registrada. Además de lo público, busca la mejor ruta, guarda su historial y un destino favorito, deja comentarios y edita su perfil.
Administrador	Gestiona la información del sistema: crea, edita, suspende y elimina rutas, responde los comentarios y consulta las métricas de uso. El rol existe en la base de datos (USER o ADMIN); sus funciones se implementan en incrementos posteriores.

2.4 Alcance del proyecto

Alcance geográfico: las rutas urbanas de Fusagasugá, la ruta a Chinauta y las intermunicipales Fusagasugá–Pasca y Fusagasugá–Arbeláez. Regla de frontera: toda ruta incluida debe tocar Fusagasugá.

Funcionalidades que se incluirán:

- Registro e inicio de sesión de usuarios, y ver y editar el perfil.
- Catálogo público de rutas, con los barrios por los que pasa cada una y el costo del pasaje.
- Búsqueda de la mejor ruta entre un origen y un destino, con el punto de abordaje más cercano, y modo offline básico sin tiempo de llegada.
- Aviso de congestión histórica, historial de búsquedas y destino favorito.
- Caja de comentarios del usuario y respuesta del administrador.
- Gestión de rutas por el administrador (crear, consultar, modificar, eliminar), suspensión temporal con notificación dentro de la app y métricas de uso.

Funcionalidades que no harán parte del proyecto:

- Seguimiento de la buseta en tiempo real (GPS en vivo): requeriría hardware en cada bus y es demasiado complejo para esta primera versión.
- Pagos en línea y notificaciones push.
- Rutas que no toquen Fusagasugá y el resto del Sumapaz (Silvania, Granada, Tibacuy, Venecia): quedan para una fase avanzada.
- Recuperación de contraseña y tarifas diferenciales (estudiante, adulto mayor).

3. Enfoque metodológico del proyecto

3.1 Análisis del contexto del proyecto

- **¿Los requerimientos pueden cambiar durante el proyecto?** Sí, y ya cambiaron: recibimos feedback del comité y del docente, el modo offline, las tarifas, el aviso de congestión y las notificaciones se redefinieron más de una vez, y el 15 de septiembre apareció un requisito nuevo (catálogo público).

- **¿Existe incertidumbre técnica?** Alta. Es la primera vez que manejamos datos geográficos (GeoJSON) y mapas, y también la primera vez del equipo con Spring Boot y con Angular; además dependemos de servicios externos y de planes gratuitos (Google Maps, Render, Supabase).
- **¿Qué riesgos iniciales identificamos?** La calidad de los datos de las rutas que consigamos, la falta de experiencia en el stack y la disponibilidad real de dos personas con otras materias (ver sección 7).
- **¿El cliente podría modificar prioridades?** Sí. El docente actúa como comité y redefine qué se entrega en cada sustentación; por eso el Sprint 2 se reestructuró la semana del comité.

3.2 Enfoque metodológico seleccionado

Elegimos un enfoque iterativo–incremental, con Scrum como marco de trabajo. El enfoque dice cómo crece el producto (por incrementos demostrables, cada uno usable de punta a punta) y Scrum dice cómo se organiza el equipo para producirlos. La base es de linaje RUP — arquitectura definida temprano, que es la hexagonal ya comprometida— y la evolución hacia ágil se justifica por la incertidumbre real descrita arriba.

- **Por qué lo elegimos:** no tenemos todo el camino trazado desde el principio y necesitamos entregar partes del sistema para validar que vamos por buen camino.
- **Ventajas:** cada incremento se puede demostrar y corregir antes de seguir; la arquitectura queda fija y el alcance funcional se ajusta con feedback.
- **Qué problemas ayuda a evitar:** hacer todo el sistema y descubrir al final que no era lo que se necesitaba, y planear en grande sobre información que todavía no se tiene.

3.3 Uso de Scrum como marco de trabajo

Organizamos el trabajo en sprints de una semana, de martes a lunes, de modo que a cada sustentación quincenal lleguen dos sprints cerrados con su informe. Los eventos son:

Evento	Cuándo	Cómo lo aplicamos
Sprint Planning	Martes 18:00	Al salir de la clase de 16–18, decidimos qué historias del backlog se atacan en el sprint.
Daily Scrum	Diario	Lo sustituimos por un check-in escrito (ayer / hoy / bloqueos): dos personas que comparten tres clases semanales no sostienen una reunión diaria. Lo dejamos por escrito para no prometer un evento que no se cumple. Además hay una sesión de pareja los jueves a las 13:00.
Sprint Review	Lunes en la noche	Cierre de sprint: mostramos lo terminado y registramos el resultado en el informe.
Sprint Retrospective	Lunes en la noche	Revisamos qué mejorar en la forma de trabajar; queda en el mismo informe.

Backlog: lo llevamos en Jira (proyecto SCRUM), organizado en 5 épicas que son a la vez los 5 incrementos demostrables (cuenta de usuario; búsqueda de ruta; historial y favoritos;

comentarios; administración), con 15 historias de usuario y 7 requisitos no funcionales, priorizados por la Product Owner. Cada historia se vincula a su rama, commit y pull request por la key SCRUM-N. Los roles son:

Persona	Capa de proceso (Scrum)	Capa técnica (docente)
Angélica	Product Owner	Responsable de Frontend y de Documentación
Santiago	Scrum Master	Líder Técnico y Responsable de Backend
Ambos	Development Team	Responsable de Pruebas (compartido)

"Responsable" significa quien responde ante el comité y revisa los PR de esa área, no quien la programa solo: ambos programamos frontend y backend. La capa técnica debe rotar cada dos sprints; esa rotación todavía no ha ocurrido y se registrará al cierre del Sprint 3.

4. Identificación y clasificación de requerimientos

4.1 Requerimientos funcionales

La guía pide entre 8 y 12 requerimientos funcionales. Agrupamos las 15 historias de usuario del backlog aprobado en 12 requerimientos; la correspondencia exacta con cada historia y su key de Jira está en el Anexo C. Todos inician con "El sistema debe...".

Código	Nombre del requerimiento	Descripción	Tipo de usuario
RF1	Registro de usuario	El sistema debe permitir que nuevos usuarios se registren con nombre, correo y contraseña.	Usuario
RF2	Inicio de sesión	El sistema debe permitir que los usuarios inicien sesión con su correo y contraseña y mantener la sesión durante una semana.	Usuario
RF3	Perfil	El sistema debe permitir ver y editar los datos personales, incluida la contraseña.	Usuario
RF4	Catálogo público de rutas	El sistema debe mostrar a cualquier visitante, sin cuenta, el listado de rutas activas y el detalle de cada una (barrios y tarifa).	Visitante
RF5	Buscar la mejor ruta	El sistema debe sugerir la mejor ruta de busetas entre un origen y un destino, mostrando los barrios por los que pasa, el costo del pasaje y el punto de abordaje más cercano.	Usuario
RF6	Aviso de congestión	El sistema debe avisar si el trayecto sugerido suele estar congestionado a la hora de la consulta.	Usuario
RF7	Historial y destino favorito	El sistema debe guardar el historial de búsquedas y permitir marcar un destino favorito que se pre-carga al iniciar la búsqueda.	Usuario
RF8	Comentarios	El sistema debe permitir dejar un comentario o sugerencia, editable mientras no haya sido respondido.	Usuario
RF9	Gestionar rutas	El sistema debe permitir al administrador crear, consultar, modificar y eliminar (de forma lógica) rutas.	Administrador
RF10	Suspender rutas	El sistema debe permitir suspender una ruta temporalmente y notificar dentro de la app a quienes la usan seguido.	Administrador

Código	Nombre del requerimiento	Descripción	Tipo de usuario
RF11	Responder comentarios	El sistema debe permitir al administrador leer y responder los comentarios de los usuarios.	Administrador
RF12	Métricas de uso	El sistema debe mostrar al administrador cinco métricas de uso de la aplicación.	Administrador

4.2 Requerimientos no funcionales

Definimos siete requerimientos no funcionales, cada uno con su instrumento de medición: no comprometemos ningún número que no podamos verificar.

Categoría	Descripción	Criterio de validación
Rendimiento	El sistema debe responder las consultas de ruta con un p95 menor a 8 s extremo a extremo en red 4G típica, con caché de 5 min por par origen–destino; el bundle GeoJSON de la red completa debe pesar ≤ 1 MB gzip.	Spring Boot Actuator + Micrometer (p95), Lighthouse en CI (LCP $< 2,5$ s) y tamaño del bundle verificado en el build.
Disponibilidad	El backend (Render, tier gratis) y el frontend (Vercel) deben responder correctamente en al menos el 95 % de los latidos en horario hábil (lun–vie 7:00–21:00), incluido el cold start; RPO < 24 h. No se compromete un servidor siempre activo.	UptimeRobot (latido cada 5 min) + backup diario de PostgreSQL y restore probado al cierre de cada sprint.
Mantenibilidad	El backend debe mantener la separación estricta de capas del hexágono, con la lógica de negocio sin depender de frameworks: 0 violaciones y al menos 70 % de cobertura.	ArchUnit en CI + JaCoCo con umbral bloqueante.
Usabilidad	El flujo de búsqueda de ruta debe completarse en máximo 4 pasos y el login pedirse como máximo una vez por semana; éxito ≥ 80 % al primer intento.	Prueba con al menos 5 usuarios externos al equipo antes de cada sustentación.
Seguridad	El sistema no debe tener vulnerabilidades altas o críticas (CVSS ≥ 7) sin suprimir; toda supresión lleva justificación, razón de no actualizar y fecha de revisión.	OWASP Dependency-Check (failBuildOnCVSS=7) + análisis estático en CI, en cada Pull Request.
Seguridad (contraseñas)	El sistema debe almacenar las contraseñas con hash bcrypt (factor de costo ≥ 10), nunca en texto plano.	Test de integración: el hash guardado no es igual a la contraseña y el login funciona con la original.
Compatibilidad	La aplicación debe funcionar en Chrome (Android), Safari (iOS) y un navegador de escritorio.	Prueba manual en dispositivos físicos y emulador antes de cada sustentación.

4.3 Priorización de requerimientos

Código	Requerimiento	Prioridad
RF1	Registro de usuario	Alta
RF2	Inicio de sesión	Alta
RF3	Perfil	Media
RF4	Catálogo público de rutas	Alta
RF5	Buscar la mejor ruta	Alta

Código	Requerimiento	Prioridad
RF6	Aviso de congestión	Alta
RF7	Historial y destino favorito	Media
RF8	Comentarios	Media
RF9	Gestionar rutas	Alta
RF10	Suspender rutas	Alta
RF11	Responder comentarios	Media
RF12	Métricas de uso	Media

Criterio: es de prioridad alta lo que, si falta, el sistema no cumple su propósito de informar y ayudar a elegir ruta (cuenta, catálogo, búsqueda, congestión) y lo que el administrador necesita para que esa información sea vigente (gestión y suspensión de rutas). Es de prioridad media lo que mejora la experiencia o la operación pero no impide usar el sistema (perfil, historial, comentarios, métricas). Ninguno quedó en baja prioridad porque lo que iba a serlo se sacó del alcance. Frente al V1 cambiaron el registro (de media a alta, porque sin cuenta no hay sesión ni historial) y el perfil (de baja a media). La prioridad del catálogo público (RF4) la asignamos nosotros al agregarlo y queda sujeta a confirmación de la Product Owner.

5. Dependencias del proyecto

Dependencia	Descripción	Impacto
Base de datos en Supabase	PostgreSQL administrado, un proyecto por ambiente (DEV, PRE, PROD). El tier gratis permite 2 proyectos por cuenta, así que el tercero va en una segunda cuenta.	Alto
Google Maps (Directions API)	Cronometrar las rutas candidatas para RF-04. Sin integrar todavía; es un puerto de salida futuro (TravelTimeProvider).	Alto
Render y Vercel (tier gratis)	Despliegue de backend y frontend (Sprint 3). Render se duerme a los 15 min de inactividad.	Alto
Datos de las rutas	Mapas oficiales y trazados de las empresas de transporte; posiblemente haya que trazar las rutas a mano. Hoy hay 8 rutas sembradas con barrios y tarifas, sin trazado GeoJSON.	Alto
Tiempo del equipo	Dos personas con otras cinco materias; el Sprint 1 cerró con velocidad 0.	Alto
CI en GitHub Actions y clave de la NVD	Los gates de calidad y seguridad corren en CI; OWASP Dependency-Check necesita una NVD_API_KEY gratuita.	Medio
Jira y GitHub	Backlog, sprints y trazabilidad de rama, commit y PR por la key SCRUM-N.	Medio

6. Configuración tecnológica del proyecto

Componente	Tecnología
Frontend	Angular 21 · TypeScript 5.9 · Node 22 · pruebas con Vitest
Backend	Java 25 · Spring Boot 3.5.16 · arquitectura hexagonal (puertos y adaptadores)

Componente	Tecnología
Autenticación	Spring Security + JWT (propio del backend, no Supabase Auth) · hash BCrypt
Base de datos	PostgreSQL 17 en Supabase · esquema versionado con Flyway (V1 a V4) · Hibernate solo valida (ddl-auto=validate)
Calidad y seguridad	ArchUnit · JaCoCo (umbral 0,70) · OWASP Dependency-Check · Checkstyle
Integración continua	GitHub Actions (mvn -B verify en el backend)
Control de versiones	Git y GitHub: tres repositorios (material del curso, backend, frontend), trunk-based con Pull Request obligatorio
Gestión del proyecto	Jira (proyecto SCRUM)
Despliegue (planificado, Sprint 3)	Frontend en Vercel · backend en Render (tier gratis)
Mapas (planificado, Sprint 4 en adelante)	Google Maps Directions API, solo para cronometrar rutas

Regla de versiones: el proyecto arranca sobre la versión vigente de cada herramienta. Spring Boot se queda en la última 3.5 y no salta a la línea 4 porque es la primera vez del equipo con Spring y el material para resolver errores es de 3.x. Cualquier salto de versión mayor es una decisión propia con su tarjeta. El estado real de las dependencias está en la evidencia de dependencias.

7. Riesgos del proyecto

Los niveles son una evaluación del equipo a la fecha del documento; entre paréntesis se indica cómo va cada riesgo.

Riesgo	Descripción	Nivel
Datos inconsistentes	Que la información de las rutas no sea real o esté mal, o que sea difícil conseguir los mapas oficiales de las empresas de transporte. (Hoy hay 8 rutas sembradas con barrios y tarifas; los trazados GeoJSON siguen pendientes.)	Alto
Falta de experiencia técnica	Es la primera vez del equipo con Spring Boot, Angular y mapas. (Se materializó en parte: el Sprint 1 cerró con velocidad 0.)	Alto
Carga del equipo	Dos personas con otras cinco materias frente a un alcance amplio. (El Sprint 2 cerró en 17 puntos según Jira; una sola medición no es capacidad.)	Alto
Entorno de producción inexistente	PROD no está desplegado: el backend corre desde un portátil. (Se despliega en el Sprint 3.)	Alto
Deuda técnica	Tener problemas al integrar la API de mapas. (Sin integrar todavía; se aborda desde el Sprint 4.)	Medio
Cold start de Render	La primera petición tras 15 minutos de inactividad tarda de 30 a 60 s y puede empujar el p95 por encima de 8 s. (Sin resolver.)	Medio
Vulnerabilidades sin corrección abierta	Hay CVE de Spring de CVSS ≥ 7 sin versión corregida publicada en abierto. (2 bloques de supresión con 15 CVE, todos con revisión el 15/01/2027.)	Medio
Cambio de alcance	Que nos pidan más cosas de las que podemos hacer o cambien prioridades. (Materializado: se redefinieron requisitos y se agregó el catálogo público.)	Medio

8. Diseño de base de datos

El diseño de la persistencia está pensado para cumplir la Tercera Forma Normal (3FN), evitando redundancias, sobre PostgreSQL 17 en Supabase. El esquema lo crea y versiona Flyway (db/migration, migraciones V1 a V4) y Hibernate solo lo valida. El modelo refleja el código real: cuatro tablas. Las tablas del V1 que dependen de funcionalidades aún no implementadas (historial de búsquedas, favoritos y comentarios) se agregarán con sus historias.

8.1 Modelo entidad–relación

Modelo entidad–relación (esquema Flyway V1–V4)

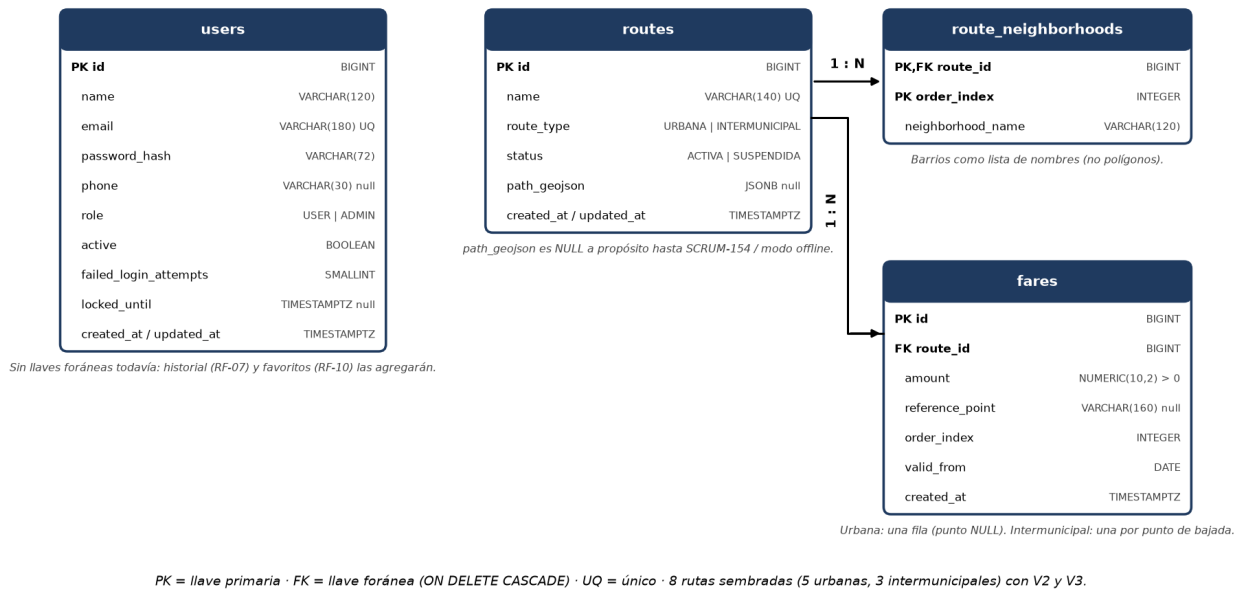


Figura 1. Modelo entidad–relación del esquema V1–V4.

8.2 Modelo relacional

Tabla	Descripción
users	Usuario final o administrador. Autenticación propia; la contraseña se guarda solo como hash BCrypt.
routes	Ruta de buseta: recorrido directo sin transbordos. Ida y vuelta son dos registros distintos. Estado ACTIVA o SUSPENDIDA.
route_neighborhoods	Barrios por los que pasa cada ruta, en orden de recorrido. Se guardan como lista de nombres, no como polígonos.
fares	Tarifa del pasaje. Urbana: una fila sin punto de referencia. Intermunicipal: una fila por punto de bajada. Lleva fecha de vigencia.

Relaciones: una ruta tiene muchos barrios y muchas tarifas (ambas con borrado en cascada). La tabla users no tiene llaves foráneas todavía: las tendrán cuando se implementen el historial (RF-07) y el destino favorito (RF-10). Las columnas failed_login_attempts y locked_until ya existen para el bloqueo por intentos fallidos, que se implementa en SCRUM-155.

8.3 Diccionario de datos

Tabla	Campo	Tipo de dato	Descripción
users	id	BIGINT	Identificador (llave primaria, autogenerado)
users	name	VARCHAR(120)	Nombre del usuario
users	email	VARCHAR(180)	Correo; único
users	password_hash	VARCHAR(72)	Hash BCrypt de la contraseña
users	phone	VARCHAR(30)	Teléfono (opcional)
users	role	VARCHAR(20)	USER o ADMIN (por defecto USER)
users	active	BOOLEAN	Cuenta activa
users	failed_login_attempts	SMALLINT	Contador de intentos fallidos (reservado para SCRUM-155)
users	locked_until	TIMESTAMPTZ	Fin del bloqueo temporal (reservado para SCRUM-155)
users	created_at / updated_at	TIMESTAMPTZ	Auditoría
routes	id	BIGINT	Identificador
routes	name	VARCHAR(140)	Nombre de la ruta; único
routes	route_type	VARCHAR(20)	URBANA o INTERMUNICIPAL
routes	status	VARCHAR(20)	ACTIVA o SUSPENDIDA
routes	path_geojson	JSONB	Trazado GeoJSON (LineString); NULL hasta que se carguen los trazados
routes	created_at / updated_at	TIMESTAMPTZ	Auditoría
route_neighborhoods	route_id	BIGINT	Ruta a la que pertenece (llave foránea, parte de la llave primaria)
route_neighborhoods	order_index	INTEGER	Posición en el recorrido (≥ 0)
route_neighborhoods	neighborhood_name	VARCHAR(120)	Nombre del barrio; único por ruta
fares	id	BIGINT	Identificador
fares	route_id	BIGINT	Ruta (llave foránea)
fares	amount	NUMERIC(10,2)	Valor del pasaje; mayor que 0
fares	reference_point	VARCHAR(160)	Punto de bajada (NULL en rutas urbanas)
fares	order_index	INTEGER	Orden de la tarifa dentro de la ruta
fares	valid_from	DATE	Fecha desde la que rige la tarifa
fares	created_at	TIMESTAMPTZ	Auditoría

9. Conclusiones del proyecto

Hacer este documento nos ayudó a aterrizar la idea de FusaRoute. Aprendimos que la ingeniería de software no es solo programar, sino planear bien los requerimientos y los riesgos para no perder tiempo, y que la calidad se diseña: escribir cada métrica con su instrumento nos obligó a reconocer que varias de las primeras eran imposibles de medir, y bajar la vara con razones fue mejor que sostener un número que no podíamos verificar. También aprendimos que un incremento no existe hasta que está mergeado: en el Sprint 1 había trabajo hecho que ante el comité valía cero.

Los mayores desafíos han sido manejar los datos de los mapas, la primera vez con Spring Boot y Angular, y decisiones de infraestructura (Render, Supabase, versiones) que condicionan qué se puede prometer. Con un enfoque iterativo estamos resolviendo los problemas poco a poco; la velocidad de 17 puntos del Sprint 2 no es una capacidad demostrada, es el resultado de un sprint concentrado sobre trabajo ya adelantado.

Aplicamos ingeniería de software con arquitectura hexagonal verificada automáticamente (ArchUnit), principios SOLID con evidencia en el código, Scrum para organizarnos y no dejar todo para el final, y trazabilidad completa entre requisito, historia, rama, commit y pull request. Lo que sigue pendiente —despliegue, búsqueda con mapas, mediciones de rendimiento y usabilidad— está declarado, no oculto.

Los anexos amplían el documento con el detalle que no entra en la estructura pedida.

Anexo A. Registro de cambios V1 → V2

Este V2 corrige y amplía el Documento V1 entregado el 16 de septiembre de 2026, con la misma estructura del docente. Desde esa fecha se aprobó el backlog completo (15 historias y 7 requisitos no funcionales), se decidió el despliegue real y se implementó el primer incremento, por lo que varias afirmaciones del V1 dejaron de ser ciertas. Cada cambio queda registrado con su razón, porque ante el comité cambiar de decisión con argumentos suma y cambiar sin registrar el porqué resta.

De qué cambió el V1 al V2

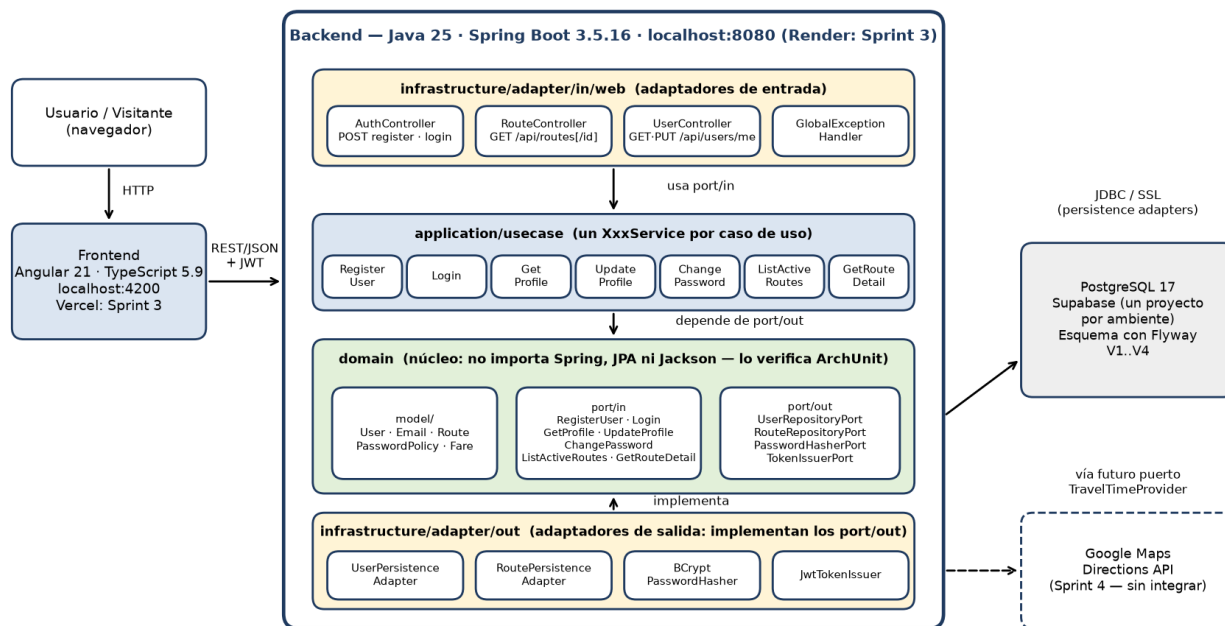
Tema	Documento V1 (16/09)	Documento V2 (28/09)	Razón
Sprints	2 semanas	1 semana, de martes a lunes	Que a cada sustentación quincenal lleguen dos sprints cerrados con su informe.
Daily Scrum	Daily Scrums cortos	Check-in escrito (ayer / hoy / bloqueos)	Dos personas con tres clases semanales en común no sostienen una reunión diaria.
Backlog	GitHub Projects	Jira, proyecto SCRUM	Trazabilidad rama-commit-PR por la key SCRUM-N.
Usuarios	2 (usuario y administrador)	3: se agrega el Visitante sin cuenta	El catálogo público de rutas (RF-15) no exige sesión.
Requerimientos funcionales	10 (RF1 a RF10)	15 (RF-01 a RF-15)	Backlog aprobado el 15/09. Ver rutas y Buscar ruta pasan a RF-15 y RF-04; agregar, editar y eliminar rutas se unifican en RF-11; nuevos RF-05 a RF-10, RF-12 a RF-14. El filtro por categoría no forma parte del backlog aprobado.
Prioridades	Registro Media; Perfil y Filtros Baja	Registro Alta; Perfil Media; ninguno Baja	Lo de baja prioridad se sacó del alcance.
Rendimiento	Mapa en menos de 3 s	p95 < 8 s extremo a extremo y LCP < 2,5 s	El cálculo de ruta depende de Google Maps; 3 s no era medible ni realista.
Disponibilidad	99 % del tiempo	95 % de los latidos en horario hábil, con cold start	El tier gratis de Render se duerme a los 15 minutos; 99 % no se puede sostener.
Escalabilidad	RNF de escalabilidad con pruebas de estrés	No figura entre los 7 RNF aprobados	Una métrica por atributo ISO/IEC 25010 y sin instrumento no se compromete.
Base de datos	MySQL; PostgreSQL 16 en la sección 8	PostgreSQL 17 en Supabase, con Flyway	Decisión de arquitectura vigente; un solo motor en todo el documento.
Modelo de datos	7 tablas: usuario, ruta, ubicacion, ruta_parada, busqueda, comentario, favorito	4 tablas implementadas: users, routes, route_neighborhoods, fares	En Fusagasugá no hay paradas fijas (la buseta se para con la mano): los barrios son una lista de nombres. Historial, favoritos y comentarios se agregarán con sus historias.
Dependencias	Base de datos, Leaflet/Google Maps, Git/GitHub	Se agregan Supabase, Render, Vercel, CI y datos de las rutas; Google Maps solo cronometra	Reflejar las dependencias reales del despliegue.
Riesgos	3 riesgos	8, con su estado real	Varios ya se materializaron (Sprint 1 con velocidad 0).

Cambios posteriores al 24/08 que ya estaban en la Actividad 3

Fecha	Cambio	Razón
24/08/2026	Actividad 3 v3: 5 métricas de calidad (una por atributo ISO/IEC 25010), cada una con instrumento ejecutable.	Una revisión interna encontró que 7 de las 8 métricas de la v2 estaban mal o medianamente definidas y 1 era redundante.
14/09/2026	RNF-02 rediseñado: frontend en Vercel y backend en Render (tier gratis), sin promesa de servidor 24/7.	El tier gratis de Render se duerme a los 15 minutos y tarda 30 a 60 s en reactivarse; prometer un RTO de servidor siempre activo era falso.
15/09/2026	Se agregó RF-15 (catálogo público de rutas) y el criterio de última milla en RF-04. RNF-01 recibió un tope de 1 MB gzip para el GeoJSON.	El catálogo completo solo lo veía el administrador, lo que contradice la justificación del proyecto (información pública clara).
21/09/2026	Sprint 2 reestructurado para el comité; SCRUM-13 y SCRUM-19 se partieron (SCRUM-155 y SCRUM-154). Los tres ambientes corren sobre Supabase y el esquema se versiona con Flyway.	El Sprint 1 cerró con velocidad 0 y no existía ninguna funcionalidad implementada en main.
27/09/2026	RNF-05 corregido: cero vulnerabilidades altas o críticas sin suprimir, con las supresiones justificadas y fechadas.	La cifra "0 detectadas" era inalcanzable: hay CVE de Spring sin versión corregida publicada en abierto.
28/09/2026	Este V2: estructura completa del docente; estado real de implementación; arquitectura, modelo de datos y contrato de API tomados del código; RF-03 adelantado al Sprint 2 (implementado en rama, sin merge).	Documentar el sistema tal como está, separando lo implementado de lo planificado.

Anexo B. Arquitectura del sistema

FusaRoute — arquitectura (estado al 28 de septiembre de 2026)



Línea continua: existe hoy · línea punteada: planificado, aún no implementado · La dependencia entre capas apunta siempre hacia el dominio.

Figura 2. Arquitectura hexagonal del backend y su contexto (estado al 28/09/2026).

El backend sigue una arquitectura hexagonal: el dominio define las interfaces (puertos) y la infraestructura las implementa (adaptadores). La dependencia apunta siempre hacia el dominio y ArchUnit lo verifica en cada build (dos reglas: capas hacia adentro y dominio sin Spring, JPA ni Jackson).

La guía del docente prescribe Controller → Service → Repository. Hexagonal la contiene, con otros nombres y la dependencia invertida:

Guía del docente	FusaRoute (hexagonal)
Controller	infraestructura/adaptador/in/web/
Service	application/usecase/ y domain/model/
Repository	infraestructura/adaptador/out/persistence/ (su interfaz vive en domain/port/out/)

Lo que hexagonal agrega es que la interfaz del repositorio pertenece al dominio: por eso se puede cambiar de base de datos sin tocar la lógica de negocio y por eso los casos de uso se prueban sin levantar Spring. La evidencia de los cinco principios SOLID, con archivo y línea, está en evidencia-solid.md.

Decisiones de arquitectura vigentes

Decisión	Estado
Backend Java 25 + Spring Boot 3.5, hexagonal; frontend Angular 21 + TypeScript 5.9	Vigente
PostgreSQL en Supabase, uno por ambiente; esquema con Flyway; Supabase solo como base administrada (sin Auth, Storage ni Realtime)	Vigente
Autenticación propia con Spring Security + JWT de una semana	Vigente
Cálculo de ruta por simulación en Google Maps con caché de 5 min	Vigente, sin implementar
Modo offline por GeoJSON con cálculo por distancia geométrica en el dominio, sin PostGIS	Vigente, sin implementar
Última milla: punto más cercano sobre el trazado y distancia a pie aproximada, sin ruteo peatonal de Google	Vigente, sin implementar
Despliegue: frontend en Vercel, backend en Render (tier gratis)	Decidido, se despliega en el Sprint 3

Cambio a sustentar ante el comité: el borrador inicial justificaba Supabase en parte por su módulo de login. Se decidió programar la autenticación en el backend porque el docente exige que ambos integrantes aprendan todo el stack, porque mantiene el dominio desacoplado de un proveedor (coherente con hexagonal) y porque da material real que defender en calidad y seguridad.

El diagrama de componentes editable (draw.io, con la plantilla del curso) está en docs/comite-2/evidencias/diagrama-arquitectura.drawio.html; se abre en un navegador con internet y se edita en app.diagrams.net.

Anexo C. Historias de usuario, casos de uso y correspondencia con los requerimientos

C.0 Correspondencia entre los requerimientos del documento y el backlog

Requerimiento del documento	Historias del backlog (RF)	Keys de Jira
RF1 Registro de usuario	RF-01	SCRUM-12
RF2 Inicio de sesión	RF-02	SCRUM-13 (bloqueo por intentos: SCRUM-155)
RF3 Perfil	RF-03	SCRUM-14
RF4 Catálogo público de rutas	RF-15	SCRUM-19 (mapa y congestión: SCRUM-154)
RF5 Buscar la mejor ruta	RF-04, RF-05, RF-06	SCRUM-15, SCRUM-153, SCRUM-16, SCRUM-17
RF6 Aviso de congestión	RF-09	SCRUM-18
RF7 Historial y destino favorito	RF-07, RF-10	SCRUM-20, SCRUM-21
RF8 Comentarios	RF-08	SCRUM-22
RF9 Gestionar rutas	RF-11	SCRUM-24
RF10 Suspender rutas	RF-12	SCRUM-25
RF11 Responder comentarios	RF-14	SCRUM-23
RF12 Métricas de uso	RF-13	SCRUM-26

C.1 Historias de usuario

Las 15 historias fueron aprobadas por ambos integrantes el 15/09/2026 y están cargadas en Jira con su código HU_MFxx_xxx. La historia HU_MF02_001 se partió en dos (SCRUM-15 y SCRUM-153) por sobredimensionada.

Historia	RF	Jira	Estado a 28/09/2026
HU_MF01_001 Registro de usuarios	RF-01	SCRUM-12	Implementada (Done)
HU_MF01_002 Inicio de sesión	RF-02	SCRUM-13	Implementada, sin bloqueo por intentos (SCRUM-155, Sprint 3)
HU_MF01_003 Ver y editar datos personales	RF-03	SCRUM-14	Implementada en rama, sin mergear; movida al Sprint 2
HU_MF02_001 Mejor ruta por tiempo	RF-04	SCRUM-15	Pendiente
HU_MF02_001b Última milla y modo offline	RF-04	SCRUM-153	Pendiente (Sprint 4)
HU_MF02_002 Barrios de la ruta sugerida	RF-05	SCRUM-16	Pendiente (los barrios ya están en la base de datos)
HU_MF02_003 Costo del pasaje	RF-06	SCRUM-17	Pendiente (las tarifas ya están en la base de datos)
HU_MF02_004 Aviso de congestión	RF-09	SCRUM-18	Pendiente (Sprint 6)
HU_MF02_005 Catálogo público de rutas	RF-15	SCRUM-19	Implementada, sin mapa ni congestión (SCRUM-154, Sprint 6)
HU_MF03_001 Historial de búsquedas	RF-07	SCRUM-20	Pendiente
HU_MF03_002 Destino favorito	RF-10	SCRUM-21	Pendiente
HU_MF04_001 Caja de comentarios	RF-08	SCRUM-22	Pendiente
HU_MF04_002 Responder comentarios	RF-14	SCRUM-23	Pendiente
HU_MF05_001 CRUD de rutas	RF-11	SCRUM-24	Pendiente
HU_MF05_002 Suspensión temporal y notificación	RF-12	SCRUM-25	Pendiente
HU_MF05_003 Panel de métricas de uso	RF-13	SCRUM-26	Pendiente

Las cuatro historias del incremento de este sprint (HU_MF01_001, 002, 003 y HU_MF02_005) están desarrolladas sobre la plantilla del docente en docs/comite-2/HU/.

C.2 Casos de uso

Caso de uso	Historia	Actor
CU-001 Registro de usuario	HU_MF01_001	Usuario Final (sin sesión)
CU-002 Inicio de sesión	HU_MF01_002	Usuario Final (sin sesión)
CU-003 Ver y editar datos personales	HU_MF01_003	Usuario Final (con sesión)
CU-004 Catálogo público de rutas	HU_MF02_005	Visitante

Cada caso de uso, con su diagrama de caso de uso, su diagrama de flujo, la tabla de datos de entrada y salida y los requisitos no funcionales aplicables, está en docs/comite-2/CU/.

Anexo D. Contrato de la API

Los endpoints salen del código del backend (rama claude/gracious-clarke-2fqs2v, commit eba1429). La colección de Postman y un archivo .http de respaldo están en el repositorio del backend.

Método y ruta	Acceso	Solicitud	Respuesta
GET /health	Público	—	Estado del servicio
POST /api/auth/register	Público	name, email, password	201 con id, name, email, role y active del usuario creado
POST /api/auth/login	Público	email, password	200 con token, tokenType, expiresAt y los datos del usuario
GET /api/routes	Público	—	Lista de rutas activas con barrios y tarifas
GET /api/routes/{id}	Público	—	Detalle de una ruta activa
GET /api/users/me	JWT	—	Perfil: id, name, email, phone, role
PUT /api/users/me	JWT	name, email, phone	Perfil actualizado
PUT /api/users/me/password	JWT	currentPassword, newPassword	Cambio de contraseña (exige la actual)

Todo lo que no está en la lista de rutas públicas exige un token JWT en el encabezado Authorization: Bearer. Un token vencido o ausente devuelve un error JSON gestionado por el manejador de errores del backend.

Anexo E. Métricas de calidad

Una métrica por atributo ISO/IEC 25010, cada una con su instrumento. Se separa lo que ya está instrumentado de lo que todavía no se puede medir.

Atributo	Métrica comprometida	Instrumento	Estado a 28/09/2026
Rendimiento	p95 < 8 s en /api/routes/search; LCP < 2,5 s; bundle GeoJSON ≤ 1 MB gzip	Actuator + Micrometer; Lighthouse en CI	No medible: el endpoint de búsqueda no existe (Sprint 4).
Disponibilidad	≥ 95 % de latidos en horario hábil, con cold start; RPO < 24 h	UptimeRobot; backup diario de Supabase	No medible: PROD no está desplegado (Sprint 3).
Mantenibilidad	0 violaciones hexagonales; ≥ 70 % de cobertura	ArchUnit en CI; JaCoCo bloqueante	Instrumentado: 2 reglas ArchUnit y umbral 0,70 sobre domain y application en mvn verify. El porcentaje real no se midió al redactar este documento.
Usabilidad	≤ 4 acciones y éxito ≥ 80 % al primer intento	Prueba con ≥ 5 usuarios externos	No realizada.
Seguridad	0 vulnerabilidades CVSS ≥ 7 sin suprimir, con las supresiones declaradas	OWASP Dependency-Check (failBuildOnCVSS=7) + análisis estático	2 bloques de supresión que cubren 15 CVE, todos con revisión el 15/01/2027. npm audit del frontend: 0 vulnerabilidades.

Nota sobre RNF-05: el texto del requisito habla de 14 hallazgos sin versión corregida y el archivo de supresiones lista 15 identificadores CVE; la diferencia no está conciliada y se revisa en el próximo sprint.

Anexo F. Estado de avance, problemas y próximos pasos

F.1 Avance por sprint

Sprint	Fechas	Resultado
Preparación	09 al 14/09	Semana de preparación: no es un sprint y no genera informe.
Sprint 1	15 al 21/09	2 issues (6 puntos), 0 completados: velocidad 0. El trabajo técnico quedó en ramas sin mergear y la infraestructura no estaba registrada como issues.
Sprint 2	22 al 28/09	17 de 17 puntos en Done según Jira (registro, login, catálogo, RNF-03, RNF-05, RNF-06) y 8 tareas de infraestructura cerradas. El comité es el lunes 28/09 a las 2:00 pm.

F.2 Problemas encontrados

- El Sprint 1 cerró vacío sin que nadie lo notara porque la infraestructura no figuraba en Jira; se corrigió creando 9 tareas visibles.
- Cuatro subtareas figuraban como terminadas sin estarlo; se devolvieron a To Do. Propuesta para el Sprint 3, pendiente de acordar entre ambos: cerrar un issue solo con su PR mergeado y el CI en verde.
- El umbral de JaCoCo (70 %) hacía fallar el propio build porque no había pruebas reales; no se bajó el número, se escribieron las pruebas (SCRUM-163).
- El archivo de supresiones OWASP tuvo un error de sintaxis XML durante cuatro días y el job de seguridad estuvo en rojo sin que la causa fuera evidente; por eso ahora el XML se valida en un paso propio antes del escaneo.
- El frontend necesita `legacy-peer-deps=true` por un desfase de parche entre `@angular/service-worker` y `@angular/core`; es deuda declarada.

F.3 Próximos pasos

- Mergear el PR de RF-03 (SCRUM-14) a main del backend y del frontend.
- Sprint 3 (29/09 al 05/10): despliegue real en Vercel y Render con sus proyectos Supabase de PRE y PROD (SCRUM-128); bloqueo por intentos fallidos (SCRUM-155).
- Aplicar las actualizaciones de parche pendientes (ArchUnit 1.5.1, JaCoCo 0.8.15) y decidir con tarjeta propia los saltos mayores (Flyway 13, Spring Boot 4).
- Sprint 4 en adelante: búsqueda de ruta con Google Maps, última milla y modo offline, y las pruebas de rendimiento y usabilidad.
- Preguntar al docente la fecha del próximo comité: no se deduce.

Referencias

International Organization for Standardization & International Electrotechnical Commission. (2011). ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. ISO/IEC.

Schwaber, K., & Sutherland, J. (2020). La guía de Scrum. <https://scrumguides.org>

Departamento Administrativo Nacional de Estadística [DANE]. Fuente de la población regional estimada (unos 285.000 habitantes) citada en la sección 1: cita exacta pendiente de confirmar por el equipo.